String Manipulation

String char
Char [] array= string.toCharArray();

Character In a string
Char x=string.ChartAt(1);

Exclude
string = string.replaceAll("\\s+","")

String to integer
Int x= Integer.parsetInt(string);

Char to int
array[i]=Character.getNumericValue(name.charAt(i));

split a string separated by spaces
String[] parts = string.split(" ");

String capitalVersion = x.toUpperCase();

Practice Problems Java

Determine the LCM of two numbers

Example LCM:
5= multiples are 5, 10,**15**
3 = multiples are 3,6,9,12,**15**
15 is the **LCM of 5 and 3**

```
public class PracticeProblems {

   public static void main(String[] args) {
      System.out.println(findLCM(5, 3)); // expected 15
   }

   public static int findLCM (int a, int b) {
      int max = 0, min = 0, lcm = 0;
      if( a < b ) {
         min = a;
         max = b;
      } else {
         max = a;
         min = b;
      }
      for (int i = 1; i <= min ; i++) {
         int x = max * i;
         if(x%min == 0) {
            lcm = x;
            break;
         }
      }
      System.out.println("LCM of " + a + " and " + b + " is " + lcm);
      return lcm;
   }
}
```

Determine the GCF of two numbers

Example:
8 = factors are 1, 2**, 4,** 8
12 = factors are 1 , 2 , 3 , **4** , 6 , 12
GCF of 8 and 12 is 4

```
public static void main(String[] args) {
   System.out.println(findLCM(5, 3)); // expected 15
   System.out.println(findGCF(8, 12)); // expected 4
   System.out.println(findGCFA(8, 12)); // expected 4

}

public static int findGCF(int a, int b) {
   while (b != 0) {
      int temp = b;
      b = a % b;
      a = temp;
   }
   return a;
}

public static int findGCFA(int a, int b) {
   int gcf = 1;
   int limit = Math.min(a, b);
   for(int i =1; i <= limit; i++) {
      if(a%i ==0 && b%i ==0) {
         gcf = i;
      }
   }
   return gcf;
}
```

---

15 Odd number

```
public class PracticeProblems {

   public static void main(String[] args) {
      printOddNumbers();
   }

   public static void printOddNumbers() {
      int counter = 0;
      int numberOfOddNumbers = 15;
      for(int i = 1; counter <= numberOfOddNumbers; i++) {
         if(i%2 != 0) {
            System.out.println(i);
            counter++;
         }
      }
   }
}
```

Odd or Even

```
public class PracticeProblems {

   public static void main(String[] args) {
      oddOrEvenNumber(0);
   }

   public static void oddOrEvenNumber(int num) {
      if(num%2==0) {
         System.out.println(num + " is even");
      } else {
         System.out.println(num + " is odd");
      }
   }
}
```

Highest digit in a number

```java
import java.util.Scanner;

public class PracticeProblems {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter the number : ");
        int num = scanner.nextInt();
        System.out.println(highestDigitInANumber(num));
    }

    public static int highestDigitInANumber(int number) {
        int highest = 0;
        while (number > 0) {
            int digit = number % 10;
            if(digit > highest) {
                highest = digit;
            }
            number = number / 10;
        }
        return highest;
    }
}
```

Output 15 prime numbers

```java
public static void main(String[] args) {
    printPrimeNumber();
}

public static void printPrimeNumber() {
    int countedPrime = 0;
    int number = 2;

    while (countedPrime < 15) {
        int divisors  = 0;
        for(int ctr = 1; ctr <=number; ctr++) {
            if(number % ctr == 0) {
                divisors ++;
            }
        }
        if(divisors  == 2) {
            System.out.println(number);
            countedPrime++;
        }
        number++;
    }
}
```

Composite or Prime

```java
public static void main(String[] args) {
    Scanner read = new Scanner(System.in);
    System.out.print("Enter a number: ");
    int x = read.nextInt();
    int factors = 0;

    for (int ctr = 1; ctr <= x; ctr++) {
        if (x % ctr == 0) {
            factors++;
        }
    }

    if (factors == 2) {
        System.out.println("The number is prime");
    } else {
        System.out.println("The number is composite");
    }
}
```

Determine if a number is power of 2

```java
public static void main(String[] args) {
    System.out.println(isPowerOfTwo(2));
}

public static boolean isPowerOfTwo(int x) {
    if(x <= 0) return false;
    while(x%2==0) {
        x /= 2;
    }
    return x == 1;
}
```

Height and Width asterisk

```java
public static void main(String [] args){
    int height = 0, width = 0;
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter height: ");
    height = scanner.nextInt();
    System.out.println("Enter width: ");
    width = scanner.nextInt();

    for(int y = 1; y <= height; y++) {
        for(int x =1; x <=width; x++) {
            System.out.print("*");
        }
        System.out.println("");
    }
}
```

Decimal or Whole Number

```java
public static void main(String[] args) {
    Scanner read = new Scanner(System.in);
    System.out.print("Enter a number: ");
    double x = read.nextDouble();  // use double to allow decimals

    if (x % 1 == 0) {
        System.out.println("The number you entered is whole");
    } else {
        System.out.println("The number you entered is decimal");
    }
}
```

Number of Lowercase x

Given a string, compute recursively (no loops) the number of lowercase 'x' chars in the string.
countX("xxhixx") → 4, countX("xhixhix") → 3, countX("hi") → 0

```java
public static void main(String [] args){
    System.out.println(countX("xxxxTX"));
}

public static int countX(String str) {
    if (str.length() == 0) {
        return 0;
    }
    if (str.charAt(0) == 'x') {
        return 1 + countX(str.substring(1));
    } else {
        return countX(str.substring(1));
    }
}
```

Count Digit 7

Given a non-negative int n, return the count of the occurrences of 7 as a digit, so for example 717 yields 2. (no loops). Note that mod (%) by 10 yields the rightmost digit (126 % 10 is 6), while divide (/) by 10 removes the rightmost digit (126 / 10 is 12).

count7(717) → 2
count7(7) → 1
count7(123) → 0

```
public static void main(String [] args){
   System.out.println(count7(78777));
}
public static int count7(int n) {
   if (n < 10) { // base case: single digit
      return n == 7 ? 1 : 0;
   } else {
      if (n % 10 == 7) { // check rightmost digit
         return 1 + count7(n / 10); // add 1 and recurse on the rest
      } else {
         return count7(n / 10); // just recurse on the rest
      }
   }
}
```

Change Letter from another letter

Given a string, compute recursively (no loops) a new string where all the lowercase 'x' chars have been changed to 'y' chars.

changeXY("codex") → "codey"
changeXY("xxhixx") → "yyhiyy"
changeXY("xhixhix") → "yhiyhiy"

```
public static void main(String [] args){
   System.out.println(changeXY("codexxx"));
}

public static String changeXY(String str) {
   if (str.length() == 0) {
      return "";
   }
   if (str.charAt(0) == 'x') {
      return "y" + changeXY(str.substring(1));
   } else {
      return str.charAt(0) + changeXY(str.substring(1));
   }
}
```

Change pi to 3.14

Given a string, compute recursively (no loops) a new string where all appearances of "pi" have been replaced by "3.14".

changePi("xpix") → "x3.14x"
changePi("pipi") → "3.143.14"
changePi("pip") → "3.14p"

```
public static void main(String [] args){
   System.out.println(changePi("pipip"));
}

public static String changePi(String str) {
   if (str.length() < 2) {
      return str; // base case: too short to contain "pi"
   } else {
      if (str.substring(0, 2).equals("pi")) {
         // replace "pi" with "3.14" and recurse on the rest
         return "3.14" + changePi(str.substring(2));
      } else {
         // keep first char and recurse on the rest
         return str.charAt(0) + changePi(str.substring(1));
      }
   }
```

Problem Statement: Given a string s, find the length of the longest substring without repeating characters.

Example Input/Output:

- Input: "abcabcbb" → Output: 3 (substring "abc")
- Input: "bbbbb" → Output: 1 (substring "b")
- Input: "pwwkew" → Output: 3 (substring "wke")

Step	Left	Right	Current Char	Action Taken	Seen Set After Action	Window Size (right - left + 1)	maxLen
1	0	0	p	Add p	{p}	1	1
2	0	1	w	Add w	{p, w}	2	2
3	0	2	w (dup)	Remove p → left=1, still dup → remove w → left=2	{}	1	2
4	2	2	w	Add w	{w}	1	2
5	2	3	k	Add k	{w, k}	2	2
6	2	4	e	Add e	{w, k, e}	3	3
7	2	5	w (dup)	Remove w → left=3	{k, e}	3	3

```java
import java.util.HashSet;
import java.util.Set;

public class Practice {

    public static void main(String[] args) {
        System.out.println(lengthOfLongestSubstring("abcdabcdeqq")); // Expected 6
        System.out.println(lengthOfLongestSubstring("bbbbb"));    // Expected 1
        System.out.println(lengthOfLongestSubstring("pwwkew"));   // Expected 3
    }


    public static int lengthOfLongestSubstring(String s) {
        Set<Character> seen = new HashSet<>();
        int maxLen = 0, left = 0;

        for(int right =0; right < s.length(); right++) {
            while(seen.contains(s.charAt(right))) {
                seen.remove(s.charAt(right));

                left++;
            }
            seen.add(s.charAt(right));
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
```

Problem: Two Sum

Statement: Given an array of integers nums and an integer target, return the indices of the two numbers such that they add up to target. You may assume that each input would have exactly one solution, and you may not use the same element twice.

Example Input/Output:

- Input: nums = [2,7,11,15], target = 9 Output: [0,1] (because nums[0] + nums[1] = 2 + 7 = 9)
- Input: nums = [3,2,4], target = 6 Output: [1,2]
- Input: nums = [3,3], target = 6 Output: [0,1]

```java
import java.util.*;

public class TwoSum {
   public static void main (String[] args) {
      System.out.println(Arrays.toString(twoSum(new int[]{2,7,11,15}, 9))); // Expected [0,1]
      System.out.println(Arrays.toString(twoSum(new int[]{3,2,4}, 6)));     // Expected [1,2]
      System.out.println(Arrays.toString(twoSum(new int[]{3,3}, 6)));
   }

   public static int[] twoSum(int[] nums, int target) {
      Map<Integer, Integer> map = new HashMap<>();
      for(int i = 0; i < nums.length; i++){
         if(map.containsKey(target - nums[i])){
            return new int[]{map.get(target - nums[i]), i};
         }
         map.put(nums[i], i);
      }
      throw new IllegalArgumentException("No solution found");
   }
}
```

Statement: Given a string containing just the characters '(', ')', '{', '}', '[', and ']', determine if the input string is valid.

A string is valid if:

1. Open brackets are closed by the same type of brackets.
2. Open brackets are closed in the correct order.
3. Every closing bracket has a corresponding opening bracket

```java
import java.util.*;
public class ValidParentheses {

   public static void main(String[] args) {
      System.out.println(isValid("()"));      // Expected true
      System.out.println(isValid("()[]{}"));  // Expected true
      System.out.println(isValid("(]"));      // Expected false
      System.out.println(isValid("([)]"));    // Expected false
      System.out.println(isValid("{[]}"));    // Expected true
   }

   public static boolean isValid(String s) {
      Stack<Character> stack = new Stack<>();
      for (char c : s.toCharArray()) {
         if (c == '(') stack.push(')');
         else if (c == '{') stack.push('}');
         else if (c == '[') stack.push(']');
         else {
            if (stack.isEmpty() || stack.pop() != c) return false;
         }
      }
      return stack.isEmpty();
```

```
    }
}
```

Easy: Reverse a String

Problem: Write a function that takes a string and returns it reversed.

Example:

- Input: "hello" → Output: "olleh"

Starter Code:

java

```
public static String reverseString(String s) {
    // TODO: implement
    return "";
}
```

```
public class ReverseWord {

    public static void main(String[] args) {
        System.out.println(reverseWords("Hello"));
        System.out.println(reverseWordsWithStringBuilder("Hello"));
    }

    public static String reverseWords(String s) {
        char [] chars = s.toCharArray();
        int left = 0, right = chars.length - 1;
        while(left < right) {
            char temp = chars[left];
            chars[left] = chars[right];
            chars[right] = temp;
            left++;
            right--;
        }
        return new String(chars);
    }

    public static String reverseWordsWithStringBuilder(String s) {
        return new StringBuilder(s).reverse().toString();
    }
}
```

```
public class ReverseString {
    public static String reverse(String str) {
        if (str == null || str.length() <= 1) {
            return str;
        }
        return reverse(str.substring(1)) + str.charAt(0);
    }

    public static void main(String[] args) {
        String str = "Hello World";
        System.out.println(reverse(str)); // Output: dlroW olleH
    }
}
```

JAVA COLLECTIONS

```
import java.util.*;

public class ListPractice {

    public static void main(String[] args) {

        //<----LIST---->//

        //ArrayList
        ArrayList<String> names = new ArrayList<>();
        // it uses array as underlying structure. if you add a new item in arraylist it creates
a new array, store the items from old to new larger array
        // ArrayList works better for random accessing item, iterating and storing/add item to
list, low remove ops
        names.add("Ariel");
        names.add("Jay");
        names.add("Fuentes");

        System.out.println("ARRAYLIST");
        names.forEach(name -> System.out.println(name));

        System.out.println("SORTED ARRAYLIST via Collections.sort");
        Collections.sort(names);
        names.forEach(name -> System.out.println(name));

        LinkedList<String> namesLinks = new LinkedList<>(); //o(1) //
        // items are stored as individual nodes.
        // each node has connection to the previous and next node, so were able to traverse the
list from the first to the last and also in reverse
        // allows add or remove element on both ends like a queue
        // removing we don't have to shift the item from right to left.
        // We just have to update the connection to the nodes around of the item that is going
to be removed
        // it can act as a stack. so you can use ppush and pop(). FILO


        // each item is a node. each element has a pointer/reference to the next and previous.
It's like chain.
        // doubly linkedlist
        namesLinks.add("Ariel");
        namesLinks.add("Jay");
        namesLinks.add("Fuentes");

        System.out.println("LINKEDLIST");
        namesLinks.forEach(name -> System.out.println(name));


        //<----QUEUE---->//
        PriorityQueue<Integer> quque = new PriorityQueue<>();



        //<----MAP---->//

        // HASHMAP
        HashMap<String, Integer> unorderedPopulationCity = new HashMap<>();
        // doesn't provide guarantee of insertion.
        // populationCity.keySet();
        // populationCity.values();
        // populationCity.entrySet();
```

```java
        // if you add a duplicate key it will overwrite the old one

        unorderedPopulationCity.put("Manila", 5);
        unorderedPopulationCity.put("Brussels", 3);
        unorderedPopulationCity.put("Paris", 2);
        unorderedPopulationCity.put("Milan", 9);

        System.out.println("HASHMAP");

        for(String key: unorderedPopulationCity.keySet()) {
            System.out.println(unorderedPopulationCity.get(key));
        }

        System.out.println("LINKEDHASHMAP");

        LinkedHashMap<String, Integer> orderedPopulationCity = new LinkedHashMap<>();
        // guarantees sort of insertion
        orderedPopulationCity.put("Manila", 5);
        orderedPopulationCity.put("Brussels", 3);
        orderedPopulationCity.put("Paris", 2);
        orderedPopulationCity.put("Milan", 9);

        for(Map.Entry<String, Integer> entry : orderedPopulationCity.entrySet()) {
            System.out.println(entry.getValue());
        }

        System.out.println("TREEMAP");

        TreeMap<String, Integer> sortedPopulationCity = new TreeMap<>();
        //keep items sorted by keys
        sortedPopulationCity.put("Manila", 5);
        sortedPopulationCity.put("Brussels", 3);
        sortedPopulationCity.put("Paris", 2);
        sortedPopulationCity.put("Milan", 9);

        sortedPopulationCity.entrySet().forEach(s -> System.out.println(s.getKey() + " " +
s.getValue()));



        //<----SET---->//

        // doesn't allow duplicate values
        Set<Integer> unorderedSet = new HashSet<>();
        // unordered. use hashmap at the background
        unorderedSet.add(6);
        unorderedSet.add(5);
        unorderedSet.add(3);
        unorderedSet.add(8);
        System.out.println("HASHSET");
        unorderedSet.forEach(item -> System.out.println(item));


        Set<Integer> orderSet = new LinkedHashSet<>();
        // ordered. it follows the insertion of order
        orderSet.add(6);
        orderSet.add(5);
        orderSet.add(3);
        orderSet.add(8);
        System.out.println("LINKEDHASHSET");
        orderSet.forEach(item -> System.out.println(item));
```

```
        Set<Integer> sortedSet = new TreeSet<>();
        // sorted based on the natural ordering of the custom ordering by implementing
Comparator interface
        sortedSet.add(6);
        sortedSet.add(5);
        sortedSet.add(3);
        sortedSet.add(8);
        System.out.println("TREESET");
        sortedSet.forEach(item -> System.out.println(item));

        //<----MAKE IT SYNCHRONIZED---->//

        List<Integer> list = Collections.synchronizedList(new ArrayList<>());

        Map<String, Integer> map = Collections.synchronizedMap(new HashMap<>());

        Set<Integer> set = Collections.synchronizedSet(new HashSet<>());



    }
}
```